

LLM4FP: LLM-Based Program Generation for Triggering Floating-Point Inconsistencies Across Compilers

Yutong Wang
University of California, Davis
United States
ytwang@ucdavis.edu

Cindy Rubio-González
University of California, Davis
United States
crubio@ucdavis.edu

Abstract

Floating-point inconsistencies across compilers can undermine the reliability of numerical software. We present LLM4FP, the first framework that uses Large Language Models (LLMs) to generate floating-point programs specifically designed to trigger such inconsistencies. LLM4FP combines Grammar-Based Generation and Feedback-Based Mutation to produce diverse and valid programs. We evaluate LLM4FP across multiple compilers and optimization levels, measuring inconsistency rate, time cost, and program diversity. LLM4FP detects nearly $2.5\times$ the number of inconsistencies as the state-of-the-art tool VARITY. Notably, most of the inconsistencies involve real-valued differences, rather than extreme values like NaN or infinities. LLM4FP also uncovers inconsistencies across a wider range of optimization levels, and finds the most mismatches between host and device compilers. These results show that LLM-guided program generation improves the detection of numerical inconsistencies. In practice, numerical software and HPC developers can use LLM4FP to compare compilers and select those that provide more accurate and consistent floating-point behavior, while compiler developers can use it to identify and address subtle consistency issues in their implementations.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging; Software reliability; Compilers**; • **Computing methodologies** → **Artificial intelligence; Natural language processing; Parallel programming languages**.

Keywords

floating-point arithmetic, differential testing, random testing, Large Language Models

ACM Reference Format:

Yutong Wang and Cindy Rubio-González. 2025. LLM4FP: LLM-Based Program Generation for Triggering Floating-Point Inconsistencies Across Compilers. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3731599.3767362>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SC Workshops '25, St Louis, MO, USA*

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1871-7/2025/11
<https://doi.org/10.1145/3731599.3767362>

1 Introduction

Floating-point arithmetic is widely used in numerical software, scientific computing, and scientific simulations. However, reasoning about its behavior remains challenging. Small rounding errors, precision issues and different compiler optimizations can all lead to unexpected behaviors [12]. For example, when the same program is compiled with different compilers and/or executed on different architectures such as CPUs and GPUs, developers may observe inconsistent results, since different compilers may generate different code for the same program [24]. Such compiler-induced inconsistencies can lead to numerical bugs in widely used libraries [10], and the floating-point research community has highlighted the difficulty of ensuring numerical accuracy and reproducibility across heterogeneous computing systems [13], with correctness in scientific computing recognized as a major challenge in High Performance Computing (HPC) [11].

Despite their importance, finding numerical inconsistencies is challenging. First, the space of possible floating-point programs is huge, which is difficult to cover all the cases using manual or random testing. Second, such inconsistencies often happen under certain compiler optimization levels and combinations of floating-point operations which are hard to trigger. Third, numerical inconsistencies are usually small and do not cause a program crash, making it hard to notice. Therefore, there is a strong need for effective and efficient approaches to generate floating-point programs that help to discover numerical inconsistencies across compilers.

To the best of our knowledge, VARITY [20] is the state-of-the-art randomized differential testing framework for generating floating-point programs that trigger numerical inconsistencies across compilers and/or optimization levels. However, VARITY mainly relies on random program generation and does not make use of any domain knowledge or prior test results. Many generated programs fail to trigger any inconsistency, leading to low testing efficiency. This limits VARITY's ability to find more subtle compiler-specific numerical inconsistencies within a reasonable budget of test programs. In a parallel line of work, Large Language Models (LLMs) have recently shown abilities in generating programs that have valid syntax and rich semantics [19]. Several tools have begun using LLMs to generate test programs for compiler fuzzing [15, 28, 37, 38]. However, they mainly target general-purpose code generation, instead of focusing on floating-point programs. As a result, existing LLM-based testing approaches are not well-suited for exposing numerical inconsistencies.

In this paper, we present LLM4FP, the first LLM-based framework to generate floating-point programs specifically designed to trigger numerical inconsistencies across compilers and/or optimization levels. Our insight to use LLMs is that they implicitly capture rich

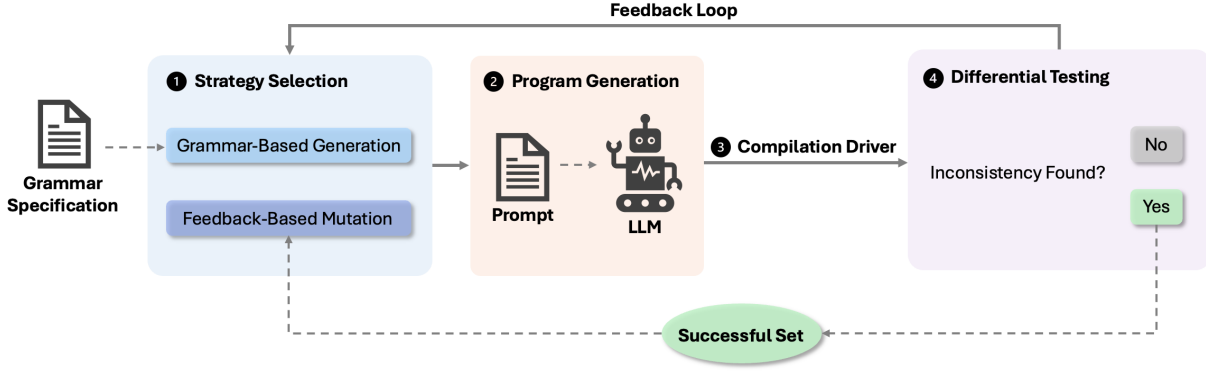


Figure 1: LLM4FP Overview

prior domain knowledge from a vast amount of source code “seen” during training. This benefits LLMs to generate programs with meaningful floating-point operations and diverse code patterns, which are often missed by existing random generators, and have the potential of exposing more compiler inconsistencies.

To realize the potential of using LLMs for floating-point program generation, we propose two complementary program generation strategies in LLM4FP, i.e., Grammar-Based Generation and Feedback-Based Mutation. The first strategy aims to explore the broad space of floating-point programs by generating new, well-formed programs from scratch without relying on previous examples. It gives us control over the structure of the generated code, ensuring that the programs are syntactically valid and semantically meaningful. However, we observe that without feedback, this strategy may lead the LLM to generate similar program patterns which limits the diversity. In contrast, the second strategy leverages feedback by mutating previously successful programs that triggered numerical inconsistencies. By showing the LLM an effective program and instructing it to generate a different one, we encourage it to explore novel yet promising programs.

We evaluate LLM4FP across multiple compilers and optimization levels. Compared with the state-of-the-art tool VARITY [20] and two LLM-based baselines, our approach consistently detects more floating-point inconsistencies while generating diverse test programs. Among all approaches, LLM4FP achieves the highest inconsistency rate of 29.33%, roughly $2.5\times$ that of VARITY, 11.93%. Concretely, LLM4FP discovers 5,280 inconsistencies out of 18,000 total pairwise comparisons across compilers and optimization levels. These improvements highlight LLM4FP’s ability to uncover subtle numerical issues that are often missed by existing methods. Furthermore, LLM4FP achieves the highest program diversity among all approaches. Notably, most of the inconsistencies involve normal and subnormal numerical values instead of extreme values such as NaN or infinities, which indicates that LLM4FP not only finds more inconsistencies, but also exposes subtler numerical divergences that are harder to detect than obvious extreme-value errors.

To summarize, this paper makes the following contributions:

- We propose LLM4FP, the first approach that uses LLMs to generate floating-point programs specifically designed to trigger numerical inconsistencies across compilers.

- We design two complementary generation strategies, i.e., Grammar-Based Generation which helps LLMs produce well-formed floating-point programs from scratch, and Feedback-Based Mutation which reuses prior successful examples to explore diverse and effective program patterns.
- We conduct a comprehensive evaluation across multiple compilers and optimization levels. LLM4FP outperforms existing approaches in both inconsistency detection rate and program diversity.

2 Technical Approach

2.1 Overview

Figure 1 shows the overall workflow of our approach, LLM4FP, for discovering floating-point inconsistencies across compilers. The first stage is to select a strategy for generating candidate programs. We propose two complementary strategies: (1) Grammar-Based Generation, which uses a given grammar specification to guide the structure of the generated programs; and (2) Feedback-Based Mutation, which randomly chooses a previously generated program that triggered numerical inconsistencies across compilers as basis to generate more programs. Once the prompt is constructed, we invoke the LLM to generate a new floating-point C program. The next stage is the Compilation Driver, which prepares the program for execution on both CPUs and GPUs. Thus this stage handles C to CUDA code translation and compilation using a configurable set of compilers and optimization flags. Finally, Differential Testing executes the programs on both host and device to detect inconsistencies in the results. If the generated program triggers any numerical inconsistencies, then it is added into the “successful” program set for potential reuse in the Feedback-Based Mutation phase. This creates a feedback loop in which successful programs help guide the LLM to generate other inconsistency-triggering programs.

2.2 Floating-Point Program Structure

Instead of directly prompting the LLM to generate numerical programs without any constraints, we specify (1) a high-level program structure to facilitate differential testing, and (2) a syntax grammar that specifies the internal structure of the numerical code to reduce the probability of generating invalid programs. Our approach

uses the LLM’s flexibility while guiding it to generate floating-point programs with a consistent structure tailored for exposing numerical inconsistencies. LLM4FP adopts the high-level program structure and grammar first introduced in VARIETY [20], which has been shown to be simple yet effective in guiding the generation of floating-point programs that expose numerical inconsistencies. The above also simplifies the comparison between LLM4FP and VARIETY in our evaluation in Section 3.

High-Level Structure. The generated program must contain only two functions: *main* and *compute*. The *compute* function takes scalar floating-point arguments, performs a sequence of arithmetic operations, and computes a scalar floating-point number as result, which is printed to standard output. The latter assumption helps when performing differential testing. This function is called from *main*.

Internal Structure. The internal structure of the function *compute* is determined by a grammar, shown in Figure 2. The grammar guides the LLM to produce meaningful floating-point computations by capturing common patterns from HPC programs [20]. These patterns are likely to influence how compilers generate and optimize code. Specifically, the grammar specification allows to indicate the floating-point precision level, either single or double precision, to use in the program. It also includes arithmetic expressions involving basic operators in {+, −, *, /}, parentheses “()” for grouping, and invoking functions from the standard C Math Library. The grammar also allows nested loops to reflect typical HPC control structures, as well as conditional branches. Additionally, the use of temporary floating-point variables is possible, which may store either scalar values or arrays. Index-based array accesses are allowed. For example, LLM4FP can generate programs that contain code patterns such as `for (i=0; i<N; i++) a[i] = ...`. Nested loops with array indexing are also supported.

2.3 LLM-Guided Program Generation

We design two strategies to guide the LLM in generating floating-point programs: Grammar-Based Generation and Feedback-Based Mutation. The first program generated is always produced using the Grammar-Based strategy, since we assume that no examples are provided to start with. Once programs that trigger inconsistencies have been found, we randomly choose between the Grammar-Based Generation and the Feedback-Based Mutation strategies for each new generation, based on a predefined probability.

2.3.1 Grammar-Based Program Generation. This strategy aims to guide the LLM to generate well-formed floating-point programs from scratch, without relying on prior examples. We carefully design the prompt to ensure that the generated floating-point programs are valid and useful for exposing numerical inconsistencies.

The prompt begins with a general instruction to create a random but valid floating-point C program. Next, the instruction includes: (1) the precision, double or single, to use for floating-point variables, (2) the program structure, including both high-level and internal structures, as described in Section 2.2, and (3) guidelines to improve robustness and code quality. Finally, the prompt instructs the LLM to output plain code only, with no formatting or explanation.

Specifically for the guidelines, we restrict the library usage to `stdio.h`, `stdlib.h`, and `math.h`. Without this constraint, the LLM

```

<function> ::= "void" "compute"
            "(" <param-list> ")" "{" <block> "}"

<param-list> ::= <param-declaration>
               | <param-list> "," <param-declaration>

<param-declaration> ::= "int" <id>
                       | <fp-type> <id>
                       | <fp-type> "*" <id>

<assignment> ::= "comp" <assign-op> <expression> ";"
               | <fp-type> <id> <assign-op> <expression> ";"

<expression> ::= <term>
               | "(" <expression> ")"
               | <expression> <op> <expression>

<term> = <identifier> | <fp-numeral>

<block> ::= {<assignment>}+
          | <if-block> <block>
          | <for-loop-block> <block>

<if-block> ::= "if" "(" <bool-expression> ")"
             "{" <block> "}"

<for-loop-block> ::= "for" "(" <loop-header> ")"
                   "{" <block> "}"

<bool-expression> ::= <id> <bool-op> <expression>

<loop-header> ::= "int" <id> ";" <id> "<" <int-numeral>
                 ";" "++" <id>

```

Figure 2: Grammar Specification for *compute* Function

may introduce unsupported or missing headers, leading to compilation failures. We also explicitly require all variables to be initialized, and instruct the LLM to avoid undefined behavior. Our approach integrates these good coding practices directly into the prompt to reduce the number of invalid programs and improve the reliability of inconsistency detection.

2.3.2 Feedback-Based Mutation. This strategy uses previously generated programs that have triggered numerical inconsistencies to create new and diverse variants. By showing the LLM an example of a successful program and explicitly instructing it to generate a different one, we encourage it to avoid repeating the same structure while still learning which patterns are effective. Our experiments confirm that this strategy helps discover more compiler inconsistencies, as well as achieves better program diversity.

The prompt first asks the LLM to change a given floating-point C program to create a new one that *behaves* differently. Next, the instruction includes: (1) the precision, double or single, to use for floating-point variables, (2) the high-level program structure, as described in Section 2.2, (3) guidelines to improve robustness and code quality, similar to those in the Grammar-Based strategy, (4) a list

of mutation strategies to consider, including reordering or deeply nesting arithmetic expressions, changing numeric constants, introducing new control flow like nested loops or conditionals, using different Math Library functions, and inserting intermediate computations, (5) a concrete previously successful example which is randomly selected from the set of successful programs that triggered inconsistencies. Finally, we include the output formatting instruction at the end of the prompt as in the last strategy. In this way, the prompt instructs the LLM to preserve key aspects of the original program, including its high-level structure, floating-point precision, and successful computational patterns, while generating novel variations.

2.4 Finding Floating-Point Inconsistencies

After generating a program, in the Compilation Driver stage, we prepare the generated program for execution on both CPUs and GPUs. Following the approach used in VARITY, we convert the generated C program into CUDA by defining the *compute* function as a `__global__` kernel, which is launched from the *main* function using a single block and a single thread. We then compile the C version using multiple host compilers, and the CUDA version using a GPU compiler, each under various optimization levels. Only binaries that compile successfully are passed to the next stage.

In the last stage, we perform differential testing [25] to check whether different compiler configurations produce inconsistent outputs for the same program and inputs. Specifically, for each program, we compare the results from the standard output for every pair of compilers under each optimization level. A *floating-point inconsistency* is recorded when two outputs are not equal in their bitwise representations, i.e., the hexadecimal encoding of the floating-point result, such as when two 64-bit doubles yield different 16-character strings. This ensures that even small floating-point differences can be detected, which may arise from compiler implementation differences, precision trade-offs, or specific optimizations. While not all inconsistencies indicate compiler bugs, detecting them is still valuable for understanding numerical instability. If a program exhibits at least one such inconsistency across any compiler pair, it is added to the successful set. These programs are then reused by the Feedback-Based Mutation strategy to generate programs that are more likely to trigger inconsistencies in future iterations.

3 Experimental Evaluation

In this section, we evaluate LLM4FP to answer the following research questions:

- RQ1** How does LLM4FP compare to our baselines in terms of effectiveness at generating floating-point programs that trigger numerical inconsistencies?
- RQ2** What kinds of floating-point inconsistencies is LLM4FP most and least effective to trigger?
- RQ3** How frequent are numerical inconsistencies between host and device compilers, or between different host compilers?
- RQ4** To what extent do different optimization levels within the same compiler introduce numerical inconsistencies when compared to the least intrusive one, i.e., `-O0_nofma`?

Table 1: Optimization Levels and Compiler Flags

Level	gcc/clang	nvcc
00_nofma	<code>-O0 -ffp-contract=off</code>	<code>-O0 -fmad=false</code>
00	<code>-O0</code>	<code>-O0</code>
01	<code>-O1</code>	<code>-O1</code>
02	<code>-O2</code>	<code>-O2</code>
03	<code>-O3</code>	<code>-O3</code>
03_fastmath	<code>-O3 -ffast-math</code>	<code>-O3 -use_fast_math</code>

3.1 Experimental Setup

3.1.1 Compilers. We consider the compilers gcc version 9.4, clang version 12.0, and nvcc version 12.3. Both gcc and clang are used as host compilers to compile CPU-targeted code, while nvcc is used as the device compiler for generating GPU-targeted code only. By default, all host compilations are linked against the GNU C Library’s Math library, while the device compiler links to the CUDA Math library. We do not include the xlc compiler (included in the evaluation of VARITY) because we do not have access to machines with the AIX operating system. However, this evaluation can be easily extended to other compilers and compiler versions.

3.1.2 Optimization Levels. To align with VARITY, we use six optimization levels across all compilers, listed in Table 1. These levels are designed to explore the impact of optimization aggressiveness and floating-point behavior triggering numerical inconsistencies. `00_nofma` minimizes compiler-induced floating-point transformations and is the most compliant with the IEEE 754 standard [24]. It uses `-O0` and explicitly disables Fused Multiply-Add (FMA), which often introduces more floating-point inconsistencies [20]. `00` uses `-O0` with FMA enabled. `01`, `02` and `03` use increasing levels of compiler optimizations. `03_fastmath` uses `-O3` with aggressive floating-point optimizations, i.e., `-ffast-math` in gcc/clang and `-use_fast_math` in nvcc, which may break IEEE 754 compliance. `03_fastmath` provides the highest performance gain, but with the lowest numerical compliance. We include this level to explore the effect of extreme optimizations on floating-point consistency, providing a contrast to more conservative levels.

3.1.3 Program Generation Budget and FP Precision. For each approach, we set a budget of 1,000 programs, each paired with a unique set of input values. Every program is compiled using supported combinations of compilers and optimization levels, and all resulting binaries are executed. Thus, for each approach there are a total of 18,000 program runs: 1,000 programs $\times$ 3 compiler pairs $\times$ 6 optimization levels. By default, the programs use double-precision floating-point arithmetic, i.e., FP64, but they could be easily extended to other precisions such as single-precision, i.e., FP32.

3.1.4 Implementation Details. LLM4FP is implemented in Python, and uses GPT-4 [29] in the feedback loop to generate floating-point programs since this model is the state-of-the-art for a wide range of NLP-based reasoning tasks [3]. Specifically, we use the gpt-4.1-2025-04-14 version of the model. We set *temperature* to 1.2 to increase randomness in token sampling and obtain more varied outputs. To reduce repetition, we set *frequency_penalty* to 0.5 to discourage the model from repeating frequently used tokens, and

presence_penalty to 0.6 to encourage new patterns in the generated code. These values are informed by findings from prior work that systematically studies hyperparameter impacts on code generation quality and diversity [2]. In addition, each new program is generated using either the Grammar-Based Generation or the Feedback-Based Mutation strategy, with probabilities of 0.3 and 0.7, respectively.

3.1.5 Environment Settings. We perform all experiments on a local machine equipped with an NVIDIA RTX A6000 GPU. The system uses the x86_64 architecture and runs Ubuntu 20.04.

3.2 RQ1: LLM4FP vs Baselines

The purpose of this RQ is to evaluate the effectiveness of LLM4FP compared to various baselines.

3.2.1 Baselines. To the best of our knowledge, we are the first to leverage LLMs to generate floating-point programs that trigger numerical inconsistencies across compilers and/or optimization levels. To evaluate the effectiveness of our proposed approach, we consider three baselines: the VARIETY framework and two LLM-based variants that differ in their generation strategy and guidance.

VARIETY [20]. A framework that triggers floating-point numerical differences in heterogeneous systems via random program generation. As discussed earlier, VARIETY uses a subset of the C language grammar to generate well-formed C/CUDA floating-point programs, and performs differential testing to find numerical inconsistencies across compilers and/or optimization levels.

DIRECT-PROMPT. We refer to our second baseline as DIRECT-PROMPT. DIRECT-PROMPT is an LLM-based variant where we directly instruct the LLM to generate valid floating-point C programs from scratch using a carefully designed prompt. In this variant, we do not provide any grammar specification or examples.

GRAMMAR-GUIDED. Compared to DIRECT-PROMPT, our third baseline, referred to as GRAMMAR-GUIDED, augments the prompt with the grammar specification described in Section 2.2, providing structural guidance to the LLM and generating programs in a more Varsity-like style. Based on GRAMMAR-GUIDED, our approach LLM4FP incorporates a feedback loop that leverages previously generated programs that triggered numerical inconsistencies. LLM4FP is guided to generate new, diverse programs that are more likely to expose other numerical inconsistencies based on sample programs that also trigger such inconsistencies.

3.2.2 Metrics. We evaluate LLM4FP and our baselines in three dimensions: effectiveness in triggering numerical inconsistencies, total time cost, and code diversity.

Inconsistency Rate and Count. To measure the effectiveness of each approach at generating floating-point programs that expose numerical inconsistencies, we define the inconsistency rate as the ratio of inconsistent output comparisons to the total number of comparisons performed across compilers and optimization levels. Let N be the number of generated programs, C be the number of compilers, and O be the number of optimization levels. Then the total number of output comparisons is:

$$\text{Total \# Comparisons} = \binom{C}{2} \times O \times N$$

For each program, we compare the outputs from every pair of compilers under each optimization level. As defined in Section 2.4, a comparison is marked *inconsistent* if the outputs differ in their bitwise representations, as captured by the hexadecimal encoding of the floating-point result. The *inconsistency rate* is then:

$$\text{Inconsistency Rate} = \frac{\# \text{ Inconsistencies}}{\text{Total \# Comparisons}}$$

Time Cost. We measure the total time taken by each approach to generate all N programs, compile each program with multiple compilers and optimization levels, and execute each compiled binary with test inputs. This is meant to capture the overall efficiency of each approach at uncovering numerical inconsistencies at scale.

Program Diversity. To evaluate the diversity of all generated programs, we use two techniques: similarity scoring [31] and code clone detection [1]. Measuring diversity is important to ensure that the LLM is producing structurally and semantically varied test programs, increasing the chance of exposing a wider range of floating-point inconsistencies.

We first compute the CodeBLEU score [31], a widely-used metric for measuring code similarity. CodeBLEU absorbs the strength of the BLEU score [30] in the n-gram match, and further incorporates code syntax via abstract syntax trees and code semantics via data-flow. A *lower* score indicates more syntactic and semantic variation, i.e., less similarity between the generated programs. Specifically, we calculate the pairwise CodeBLEU score between all N generated programs and report the average.

In addition to CodeBLEU, we analyze the programs using the NiCad Clone Detector [7] to detect redundant generation patterns that are structurally identical but superficially different, and thus unlikely to reveal different numerical inconsistencies. NiCad supports the commonly used four types of code clones, Type-1 through Type-4, as originally proposed by [32]. In our analysis, we focus on Type-1, Type-2, and Type-2c code clones. Type-1 clones are identical code fragments except for differences in whitespace or comments. Type-2 clones allow syntactic similarity with variations in identifiers, literals, and types. Type-2c, a stricter subtype specific to NiCad, corresponds to structurally identical code with only consistent renaming of identifiers. We do not consider Type-3 (clones with inserted, deleted, or reordered statements), or Type-4 (semantic clones with similar behavior but different syntax), as floating-point programs are often computation-intensive and highly sensitive to small syntactic or semantic changes. Small modifications may lead to significantly different numerical behaviors, making such code fragments of interest for potentially triggering numerical inconsistencies.

It is important to acknowledge that even programs not identified as clones may still expose the same numerical inconsistencies. To detect such cases, a more detailed analysis is necessary. Tools like pLiner [16] or Ciel [26] can be employed, since given a program that triggers a numerical inconsistency across different compilers or optimization levels, these tools pinpoint the specific lines or expressions responsible for the inconsistency. Using this information, programs can be grouped into equivalence classes based on the root causes of their numerical inconsistencies.

Table 2: Comparing LLM4FP with baselines in terms of numerical inconsistencies, time cost (hh:mm:ss), and program diversity. Incons. Rate = inconsistency rate; # Incons. = total number of inconsistencies. Lower CodeBLEU score is better.

Approach	Incons. Rate	# Incons.	Time Cost	CodeBLEU
VARITY	11.93%	2,147	00:30:42	0.3581
DIRECT-PROMPT	14.23%	2,562	04:16:37	0.3561
GRAMMAR-GUIDED	16.47%	2,964	05:56:13	0.3442
LLM4FP	29.33%	5,280	05:37:42	0.2788

3.2.3 Evaluation Results. Table 2 compares the effectiveness of LLM4FP and the baselines at generating floating-point programs that trigger numerical inconsistencies in terms of the metrics described in Section 3.2.2. Among all approaches, LLM4FP achieves the highest inconsistency rate at 29.33%, roughly 2.5× that of VARITY, 11.93%. In total, LLM4FP discovers 5,280 inconsistencies among the total 18,000 comparisons across compilers and optimization levels. This indicates that incorporating the program structural guidance via the VARITY grammar specification and a feedback loop via examples of inconsistent programs enables the LLM to generate test programs that are more likely to trigger numerical inconsistencies across compilers and/or optimization levels. GRAMMAR-GUIDED also outperforms VARITY and DIRECT-PROMPT by 4.54% and 2.24%, respectively. This suggests that grammar-guided generation combined with the LLM alone improves inconsistency detection.

In terms of the time cost, VARITY remains the most efficient approach requiring only about 30 minutes to complete the full pipeline for 1,000 generated programs, while LLM-based approaches take approximately from 4 to 6 hours. The additional cost arises from the latency of invoking the LLM API during program generation, which accounts for more than half of the total time. In the future, this overhead could potentially be reduced by batching API calls or leveraging local LLM inference to lower latency. Though LLM4FP has a longer runtime than VARITY, its improvement in inconsistency detection indicates a trade-off between testing effectiveness and time cost.

Because it is important to avoid repeatedly triggering the same numerical inconsistencies, diversity in program generation is crucial. Table 2 compares LLM4FP with the baselines in terms of the CodeBLEU score among the generated programs. A lower score is better, as it indicates more diverse programs. The results show a clear descending trend in CodeBLEU scores from VARITY (0.3581) to DIRECT-PROMPT (0.3561), GRAMMAR-GUIDED (0.3442), and finally to LLM4FP (0.2788), suggesting that program diversity consistently improves across these approaches. Among them, LLM4FP achieves the highest diversity, reducing the CodeBLEU score by 22.14% compared to VARITY. We attribute this improvement to the feedback-driven generation mechanism of LLM4FP, which iteratively mutates one existing example based on the model’s prior successes. This feedback loop encourages the LLM to explore new program variants that differ behaviorally from previous ones while retaining effective structures for triggering numerical inconsistencies. By explicitly guiding the model to refine and diversify rather than simply regenerate, LLM4FP improves the diversity of program generation while maintaining meaningful program structures.

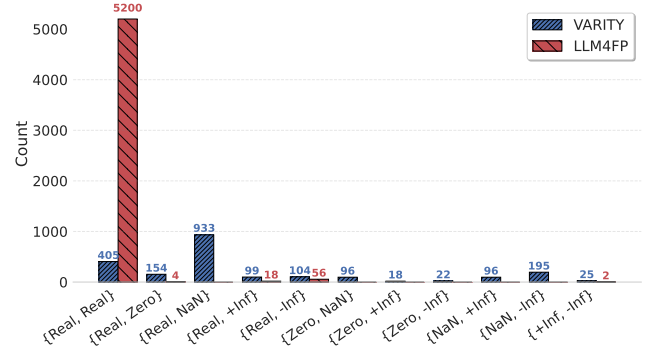


Figure 3: Inconsistency counts of different kinds between two compilers (VARITY vs. LLM4FP)

Regarding Type-1, Type-2, and Type-2c clones, none were detected for any approach, showing that the generated programs across all methods are structurally distinct. This confirms that the prompt design in our LLM-based approaches effectively avoids trivial code duplication while maximizing the discovery of unique inconsistencies.

Response to RQ1: LLM4FP achieves the highest inconsistency rate at 29.33% and the best program diversity with the lowest CodeBLEU score of 0.2788. This comes to a higher cost due to the latency of API LLM calls. On the other hand, GRAMMAR-GUIDED and DIRECT-PROMPT’s inconsistency rates are 16.47% and 14.23%, while having similar time cost as LLM4FP. Finally, VARITY achieves the lowest inconsistency rate of 11.93% but also remains the most efficient approach in terms of time cost.

3.3 RQ2: Characterizing Floating-Point Inconsistencies

3.3.1 Experimental Setup. Since LLM4FP outperforms the other LLM-based variants, we focus the rest of the evaluation on comparing LLM4FP with VARITY with respect to the floating-point inconsistencies triggered by their generated floating-point programs. In this RQ, we discuss how often different classes of floating-point inconsistencies are triggered. We follow VARITY to investigate five classes of floating-point inconsistencies: *Real* (normal and subnormal numbers), *Zero* (positive zero and negative zero), *+Inf* (positive infinity), *-Inf* (negative infinity), and *NaN* (not-a-number). Specifically, we count the number of times that there is a floating-point inconsistency of the kind $\{r_i, r_j\}$, where $r_i \neq r_j$ and both results belong to different numerical categories in $\{Real, Zero, +Inf, -Inf, NaN\}$. For example, an inconsistency between a real number and a zero is counted once as a $\{Real, Zero\}$ inconsistency.

3.3.2 Evaluation Results. Figure 3 shows a marked difference between LLM4FP and VARITY in the distribution of the classes of inconsistencies. VARITY’s inconsistencies occur across all kinds, but the majority involve extreme values like NaN and infinities. On the other hand, 98.48% of LLM4FP’s triggered inconsistencies are $\{Real, Real\}$, which are nearly 13× more than VARITY’s. Typically, $\{Real, Real\}$ inconsistencies are specially meaningful since they are

Table 3: Inconsistency counts for LLM4FP across optimization levels (“–” indicates the category did not appear)

Level	<i>Real, Real</i>	<i>Real, Zero</i>	<i>Real, +Inf</i>	<i>Real, -Inf</i>	<i>+Inf, -Inf</i>
00_nofma	720	–	–	–	–
00	840	–	–	4	–
01	830	–	–	4	–
02	814	–	–	4	–
03	798	–	–	4	–
03_fastmath	1198	4	18	40	2
Total	5,280				

much harder for developers to detect and fix. VARITY triggers many extreme values because it relies on unguided random program generation, which can produce operations that lead to obvious edge cases, such as division by zero, or invalid arithmetic. In contrast, the advantage of using the LLM is that it tends to generate more structured and semantically plausible computations that are less likely to cause such catastrophic outcomes. Instead, LLM4FP mainly triggers inconsistencies of the $\{Real, Real\}$ class, which indicate subtle numerical divergence in normal floating-point computations.

When further breaking down each inconsistency kind by optimization level for LLM4FP, we find that the $\{Real, Real\}$ inconsistency class is observed across all optimization levels, as shown in Table 3. Among them, 03_fastmath contributes the largest number of inconsistencies. The extreme values from $\{Real, +Inf\}$, $\{Real, -Inf\}$, and $\{-Inf, +Inf\}$ are relatively rare and mainly occur under 03_fastmath. The results show that the number of inconsistencies across optimization levels is relatively stable, except for 03_fastmath, which consistently amplifies inconsistencies.

Response to RQ2: The most common floating-point inconsistency triggered by the programs generated by LLM4FP is the $\{Real, Real\}$ kind, which accounts for 98.48% of all inconsistencies and is observed across all optimization levels. In contrast, extreme-value categories involving $+Inf$, $-Inf$, or NaN are rare and occur in the most aggressive optimization level, 03_fastmath.

3.4 RQ3: Inconsistencies Across Compilers

In this RQ, we analyze whether numerical inconsistencies are more likely to occur between different host compilers or between host and device compilers, and how optimization levels influence these inconsistencies. We also analyze all possible combinations of compilers to see which combinations produce the largest and smallest differences. We consider the 16 hexadecimal digits of the floating-point result, and report an inconsistency if any of them do not match. Table 4 summarizes the inconsistency rates for LLM4FP and VARITY across compiler pairs and optimization levels. In addition to the rates, it also reports the minimum, maximum, and average number of differing digits observed in inconsistent results.

For both approaches, the highest inconsistency rates consistently happen in comparisons between host and device compilers, i.e., gcc vs. nvcc and clang vs. nvcc, rather than between host compilers such as gcc vs. clang. For example, in LLM4FP, the most inconsistent pair is gcc vs. nvcc, with a total inconsistency rate of 13.01%, compared to 3.74% for gcc vs. clang. A similar trend is observed

in VARITY, where the highest rate of 5.50% is from clang vs. nvcc, much higher than that of gcc vs. clang which is 1.17%.

Optimization level plays an important role in when and how frequent numerical inconsistencies occur. While similar trends were observed in Section 3.3, here we focus specifically on inconsistencies between host and device compilers. For VARITY, the highest inconsistency rates consistently occur at the 03_fastmath level, reaching up to 2.06% for clang vs. nvcc, while the rates at other optimization levels remain very low, mostly below 1%. This is expected as aggressive optimizations and fast-math transformations can increase the chance of observing inconsistencies across compilers. For LLM4FP, the 03_fastmath level still yields the highest inconsistency rates across compiler pairs, peaking at 2.82% for clang vs. nvcc. However, unlike VARITY, other optimization levels also exhibit relatively high rates around 2%, suggesting that LLM4FP triggers inconsistencies more broadly across compiler settings. This observation highlights the potential of LLM4FP for generating more complex and diverse floating-point programs that expose subtle compiler differences beyond aggressive optimization scenarios.

Beyond inconsistency rates, the digit difference results provide additional insight into the severity of numerical discrepancies. On average, LLM4FP inconsistencies exhibit smaller digit differences, i.e., from around 1 to 3 digits, compared to VARITY, whose average differences are typically between 4 and 8 digits. This suggests that while LLM4FP exposes inconsistencies more frequently, most of them correspond to subtle numerical divergences rather than large-magnitude errors. In particular, LLM4FP mainly triggers inconsistencies of the $\{Real, Real\}$ class, reflecting sensitive variations in normal floating-point computations rather than catastrophic precision loss.

Response to RQ3: Host–device numerical inconsistencies are more frequent than host–host ones. Compared to VARITY, LLM4FP discovers more inconsistencies even at lower optimization levels, indicating its ability to expose numerical differences beyond those revealed by aggressive optimizations. LLM4FP also exposes smaller digit differences, indicating that most numerical discrepancies are subtle rather than large-magnitude errors.

3.5 RQ4: Comparing to 00 without FMA

This section compares the results of -00_nofma to all other optimization levels. Since -00_nofma is the level most compliant with IEEE, we aim to measure how much other optimization levels differ from it within each compiler. Table 5 shows the percentage of times that there is a difference between -00_nofma and each other optimization level for VARITY and LLM4FP. All comparisons are done within the same compiler across different optimization levels.

As shown in Table 5, the inconsistency rate varies when comparing 00_nofma with different optimization levels. For VARITY, the inconsistency rates at lower optimization levels from 00 to 03 remain near zero, all below 0.05%. It mainly detects differences caused by the most aggressive optimization level, 03_fastmath, especially for clang which shows an inconsistency rate of 2.07% compared to 00_nofma. In contrast, LLM4FP reports higher rates across all levels, with noticeable differences at moderate optimizations, reaching up to 0.78% for clang. The results indicate that

Table 4: Inconsistency rates and digit differences (min / max / avg) across compiler pairs at each optimization level for VARITY and LLM4FP (higher inconsistency rates are preferred)

Level	VARITY			LLM4FP		
	gcc, clang	gcc, nvcc	clang, nvcc	gcc, clang	gcc, nvcc	clang, nvcc
00_nofma	0.03%	0.67%	0.66%	0.48%	1.77%	1.75%
	(1/1/1.00)	(1/14/4.73)	(1/14/4.80)	(1/1/1.00)	(1/15/1.22)	(1/15/1.23)
00	0.03%	0.69%	0.68%	0.48%	2.12%	2.09%
	(1/1/1.00)	(1/14/4.58)	(1/14/4.64)	(1/1/1.00)	(1/15/1.25)	(1/15/1.25)
01	0.03%	0.70%	0.70%	0.46%	2.19%	1.98%
	(1/1/1.00)	(1/14/4.56)	(1/14/4.56)	(1/1/1.00)	(1/15/1.24)	(1/15/1.27)
02	0.03%	0.70%	0.70%	0.39%	2.17%	1.98%
	(1/1/1.00)	(1/14/4.56)	(1/14/4.56)	(1/1/1.00)	(1/15/1.24)	(1/15/1.27)
03	0.03%	0.70%	0.70%	0.38%	2.12%	1.95%
	(1/1/1.00)	(1/14/4.56)	(1/14/4.56)	(1/1/1.00)	(1/15/1.25)	(1/15/1.27)
03_fastmath	1.02%	1.80%	2.06%	1.55%	2.64%	2.82%
	(1/16/8.78)	(1/16/6.75)	(1/16/7.30)	(1/16/3.04)	(1/16/2.28)	(1/16/3.08)
Total	1.17%	5.26%	5.50%	3.74%	13.01%	12.57%

Table 5: Inconsistency rates between any optimization level and 00_nofma for VARITY and LLM4FP (higher inconsistency rates are preferred, and “–” indicates there is no inconsistency)

Level	VARITY			LLM4FP		
	gcc	clang	nvcc	gcc	clang	nvcc
00	–	–	0.03%	–	–	0.51%
01	0.02%	0.04%	0.03%	0.36%	0.75%	0.51%
02	0.02%	0.04%	0.03%	0.41%	0.77%	0.51%
03	0.01%	0.04%	0.03%	0.38%	0.78%	0.51%
03_fastmath	1.61%	2.07%	0.04%	1.76%	2.30%	0.52%
Total	1.66%	2.19%	0.16%	2.91%	4.60%	2.56%

LLM4FP’s programs are better at finding variations even when optimizations are less aggressive. For both VARITY and LLM4FP, clang has the highest number of floating-point inconsistencies across its optimization levels, while nvcc is the most stable. Compared to VARITY, LLM4FP finds more variations across optimization levels within each compiler.

Response to RQ4: Different optimizations within the same compiler can introduce different floating-point inconsistencies compared to 00_nofma. LLM4FP finds inconsistencies across all optimization levels, not just the most aggressive ones. clang has the most inconsistencies while nvcc is the most stable.

4 Limitations

While LLM4FP improves the state of the art in generating floating-point programs that trigger numerical inconsistencies, it also has several limitations. First, our approach has a higher runtime cost due to the latency of LLM API calls which may limit scalability for large-scale testing. Nevertheless, the improved effectiveness of LLM4FP helps offset this overhead. Second, while we explicitly

instruct the LLM to avoid undefined behavior, we cannot guarantee its absence, which may cause false positives in inconsistency detection. In the future, we plan to explore constraining our program generation rules and integrating runtime undefined behavior checkers such as GCC/Clang Undefined Behavior Sanitizer or AddressSanitizer [35] to reduce false positives while maintaining valid program semantics.

Another limitation concerns prompt design and the interpretation of detected floating-point inconsistencies. The effectiveness of LLM4FP depends heavily on carefully crafted prompts. In practice, we find that good prompts combine structural and semantic constraints with explicit formatting rules and targeted mutation strategies, often supported by a successful example, whereas poor prompts tend to be vague or unconstrained, which can result in trivial or uninformative programs. If the prompts are poorly designed, the effectiveness of triggering numerical inconsistencies will be limited. Lastly, not all inconsistencies detected by LLM4FP indicate real bugs. Some may reflect allowed differences in compiler behavior, while others can impact program stability and lead to incorrect results. In either case, prior work [16, 20, 26, 27, 33] has shown that it is important to be aware of floating-point inconsistencies across compiler and/or compiler optimizations.

5 Related Work

Compiler testing has been widely studied to ensure the correctness and reliability of compiler behavior [4]. Traditional compiler testing techniques include random testing [9, 18], randomized differential testing [14, 25], equivalence modulo inputs testing [21], program enumeration [42] among others. The main challenge in compiler testing is to generate diverse, syntactically and semantically valid programs to test the compiler behaviors. Grammar-based approaches synthesize programs from scratch by following generation rules to ensure syntactic validity, such as CSmith [39], YARP-Gen [23], and LangFuzz [17]. To generate more realistic and complicated test programs, mutation-based approaches [8, 21, 22, 42] generate new programs by mutating valid seed programs.

For floating-point programs, FLiT [33] provides a test infrastructure for detecting variability in floating-point code across compilers and platforms. Unlike program generation approaches, FLiT does not synthesize new tests but instead systematically runs existing workloads to uncover cross-platform result differences. VARIETY [20] extends compiler testing to this domain by using grammar-guided random generation to synthesize test programs, focusing on numerical inconsistencies across different compilers and architectures rather than compiler bugs. However, it does not leverage domain knowledge or feedback from previous tests, which limits its ability to discover floating-point inconsistencies efficiently. In contrast, our work uses LLMs to generate and adapt test programs based on prior results, enabling us to discover more floating-point inconsistencies across compilers.

Several prior tools address different aspects of compiler-induced numerical inconsistencies but are complementary to our program-generation focus. pLiner [16] isolates known numerical inconsistencies in C/C++ CPU programs at the line level using hierarchical code isolation and precision enhancement, while Ciel [26] extends it to heterogeneous programs and isolates inconsistencies at the expression level, enabling finer-grained analysis. Tools such as CIV [40], S3FP [6], and CIGEN [27] are input-generation methods that aim to expose floating-point inconsistencies or high-precision errors in existing programs, rather than synthesizing new programs. FPDiff [36] performs differential testing on automatically identified synonymous functions across numerical libraries to detect inconsistencies in their results under specific inputs. Together, these approaches focus on isolating or exposing inconsistencies in existing programs or libraries, whereas our work targets automatic generation of new floating-point test programs to discover inconsistencies across compilers from scratch.

Recent advances in LLMs have opened new opportunities for test program generation [19]. LLMs like Codex [5] and GPT [29] have demonstrated strong capabilities in generating syntactically correct and semantically meaningful code from natural language prompts in tools like TestPilot [34] and ChatTester [41]. Several recent works have applied LLMs to testing compilers [15, 28, 37, 38]. For example, WhiteFox [38] integrates LLMs with compiler source code to guide white-box fuzzing for optimization testing. Fuzz4All [37] uses LLMs to fuzz compilers across multiple languages via auto-prompting and iterative mutation. Unlike our work, they focus on general bug detection and do not target floating-point inconsistencies. Overall, our work bridges the gap between LLM-based program generation and numerical inconsistency detection across compilers, which is underexplored in prior research.

6 Conclusion

We propose LLM4FP, a novel LLM-based approach for generating floating-point programs that trigger numerical inconsistencies across compilers and/or optimization levels. By combining Grammar-Based Generation and Feedback-Based Mutation, LLM4FP significantly outperforms existing tools like VARIETY, detecting nearly 2.5 times as many inconsistencies. Our evaluation shows improved program diversity over other baselines, with a trade-off in time cost. Most inconsistencies it finds involve real-valued results, and they demonstrate LLM4FP's effectiveness at exposing inconsistencies

across a wide range of optimization levels and between host and device compilers. Our code, documentation and experimental data are publicly available at <https://github.com/ucd-plse/LLM4FP>.

Acknowledgments

This work was supported in part by the National Science Foundation under award CNS-2346396, and the U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research, under award DE-SC0020286.

References

- [1] Qurat Ul Ain, Wasi Haider Butt, Muhammad Waseem Anwar, Farooque Azam, and Bilal Maqbool. 2019. A Systematic Review on Code Clone Detection. *IEEE Access* 7 (2019), 86121–86144. doi:10.1109/ACCESS.2019.2918202
- [2] Chetan Arora, Ahnaf Ibn Sayeed, Sherlock A. Licorish, Fanyu Wang, and Christoph Treude. 2024. Optimizing Large Language Model Hyperparameters for Code Generation. *CoRR abs/2408.10577* (2024). arXiv:2408.10577 doi:10.48550/ARXIV.2408.10577
- [3] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrkke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott M. Lundberg, Harsha Nuri, Hamid Palangi, Marco Túlio Ribeiro, and Yi Zhang. 2023. Sparks of Artificial General Intelligence: Early experiments with GPT-4. *CoRR abs/2303.12712* (2023). arXiv:2303.12712 doi:10.48550/ARXIV.2303.12712
- [4] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2021. A Survey of Compiler Testing. *ACM Comput. Surv.* 53, 1 (2021), 4:1–4:36. doi:10.1145/3363562
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR abs/2107.03374* (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [6] Wei-Fan Chiang, Ganesh Gopalakrishnan, Zvonimir Rakamaric, and Alexey Solov'yev. 2014. Efficient search for inputs causing high floating-point errors. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '14, Orlando, FL, USA, February 15-19, 2014*, José E. Moreira and James R. Larus (Eds.). ACM, 43–52. doi:10.1145/2555243.2555265
- [7] James R. Cordy and Chanchal K. Roy. 2011. The NiCad Clone Detector. In *The 19th IEEE International Conference on Program Comprehension, ICPC 2011, Kingston, ON, Canada, June 22-24, 2011*. IEEE Computer Society, 219–220. doi:10.1109/ICPC.2011.26
- [8] Alastair F. Donaldson, Hugues Evrard, Andrei Lascu, and Paul Thomson. 2017. Automated testing of graphics shader compilers. *Proc. ACM Program. Lang.* 1, OOPSLA (2017), 93:1–93:29. doi:10.1145/3133917
- [9] Joe W. Duran and Simeon C. Ntafos. 1984. An Evaluation of Random Testing. *IEEE Trans. Software Eng.* 10, 4 (1984), 438–444. doi:10.1109/TSE.1984.5010257
- [10] Anthony Di Franco, Hui Guo, and Cindy Rubio-González. 2017. A comprehensive study of real-world numerical bug characteristics. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering, ASE 2017, Urbana, IL, USA, October 30 - November 03, 2017*, Grigore Rosu, Massimiliano Di Penta, and Tien N. Nguyen (Eds.). IEEE Computer Society, 509–519. doi:10.1109/ASE.2017.8115662
- [11] Maya B. Gokhale, Ganesh Gopalakrishnan, Jackson Mayo, Santosh Nagarakatte, Cindy Rubio-González, and Stephen F. Siegel. 2023. Report of the DOE/NSF Workshop on Correctness in Scientific Computing, June 2023, Orlando, FL. *CoRR abs/2312.15640* (2023). arXiv:2312.15640 doi:10.48550/ARXIV.2312.15640
- [12] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Comput. Surv.* 23, 1 (1991), 5–48. doi:10.1145/103162.103163
- [13] Ganesh Gopalakrishnan, Ignacio Laguna, Ang Li, Pavel Panchekha, Cindy Rubio-González, and Zachary Tatlock. 2021. Guarding Numerics Amidst Rising Heterogeneity. In *5th IEEE/ACM International Workshop on Software Correctness for HPC Applications, Correctness@SC 2021, St. Louis, MO, USA, November 19, 2021*, Ignacio Laguna and Cindy Rubio-González (Eds.). IEEE, 9–15. doi:10.1109/CORRECTNESS54621.2021.00007
- [14] Alex Groce, Gerard J. Holzmann, and Rajeev Joshi. 2007. Randomized Differential Testing as a Prelude to Formal Verification. In *29th International Conference on Software Engineering (ICSE 2007), Minneapolis, MN, USA, May 20-26, 2007*. IEEE Computer Society, 621–631. doi:10.1109/ICSE.2007.68
- [15] Qiuhan Gu. 2023. LLM-Based Code Generation Method for Golang Compiler Testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 2201–2203. doi:10.1145/3611643.3617850
- [16] Hui Guo, Ignacio Laguna, and Cindy Rubio-González. 2020. pLiner: isolating lines of floating-point code for compiler-induced variability. In *Proceedings of the*

- International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9–19, 2020*, Christine Cuicchi, Irene Qualters, and William T. Kramer (Eds.). IEEE/ACM, 49. doi:10.1109/SC41405.2020.00053
- [17] Christian Holler, Kim Herzig, and Andreas Zeller. 2012. Fuzzing with Code Fragments. In *Proceedings of the 21th USENIX Security Symposium, Bellevue, WA, USA, August 8–10, 2012*, Tadayoshi Kohno (Ed.). USENIX Association, 445–458. <https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler>
- [18] Darrel C. Ince and Sharam Hekmatpour. 1986. An Empirical Evaluation of Random Testing. *Comput. J.* 29, 4 (1986), 380. doi:10.1093/COMJNL/29.4.380
- [19] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *CoRR* abs/2406.00515 (2024). arXiv:2406.00515 doi:10.48550/ARXIV.2406.00515
- [20] Ignacio Laguna. 2020. Varsity: Quantifying Floating-Point Variations in HPC Systems Through Randomized Testing. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, New Orleans, LA, USA, May 18–22, 2020. IEEE, 622–633. doi:10.1109/IPDPS47924.2020.00070
- [21] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014*, Michael F. P. O'Boyle and Keshav Pingali (Eds.). ACM, 216–226. doi:10.1145/2594291.2594334
- [22] Vu Le, Chengnian Sun, and Zhendong Su. 2015. Finding deep compiler bugs via guided stochastic program mutation. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2015, part of SPLASH 2015, Pittsburgh, PA, USA, October 25–30, 2015*, Jonathan Aldrich and Patrick Eugster (Eds.). ACM, 386–399. doi:10.1145/2814270.2814319
- [23] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random testing for C and C++ compilers with YARPGen. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 196:1–196:25. doi:10.1145/3428264
- [24] Peter W. Markstein. 2008. The New IEEE-754 Standard for Floating Point Arithmetic. In *Numerical Validation in Current Hardware Architectures, 6.1 - 11.1.2008 (Dagstuhl Seminar Proceedings, Vol. 08021)*, Annie A. M. Cuyt, Walter Krämer, Wolfram Luther, and Peter W. Markstein (Eds.). Internationales Begegnungs- und Forschungszentrum für Informatik (IBFI), Schloss Dagstuhl, Germany. <http://drops.dagstuhl.de/opus/volltexte/2008/1448>
- [25] William M. McKeeman. 1998. Differential Testing for Software. *Digit. Tech. J.* 10, 1 (1998), 100–107. <https://www.hpl.hp.com/hpjournal/dtj/vol10num1/vol10num1art9.pdf>
- [26] Dolores Miao, Ignacio Laguna, and Cindy Rubio-González. 2023. Expression Isolation of Compiler-Induced Numerical Inconsistencies in Heterogeneous Code. In *High Performance Computing - 38th International Conference, ISC High Performance 2023, Hamburg, Germany, May 21–25, 2023, Proceedings (Lecture Notes in Computer Science, Vol. 13948)*, Abhinav Bhatele, Jeff R. Hammond, Marc Baboulin, and Carola Kruse (Eds.). Springer, 381–401. doi:10.1007/978-3-031-32041-5_20
- [27] Dolores Miao, Ignacio Laguna, and Cindy Rubio-González. 2024. Input Range Generation for Compiler-Induced Numerical Inconsistencies. In *Proceedings of the 38th ACM International Conference on Supercomputing, ICS 2024, Kyoto, Japan, June 4–7, 2024*, Kenji Kise, Valentina Salapura, Murali Annavaram, and Ana Lucia Varbanescu (Eds.). ACM, 201–212. doi:10.1145/3650200.3656618
- [28] Christian Munley, Aaron Jarmusch, and Sunita Chandrasekaran. 2024. LLM4VV: Developing LLM-driven test suite for compiler validation. *Future Gener. Comput. Syst.* 160 (2024), 1–13. doi:10.1016/J.FUTURE.2024.05.034
- [29] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). arXiv:2303.08774 doi:10.48550/ARXIV.2303.08774
- [30] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, July 6–12, 2002, Philadelphia, PA, USA*. ACL, 311–318. doi:10.3115/1073083.1073135
- [31] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. *CoRR* abs/2009.10297 (2020). arXiv:2009.10297 <https://arxiv.org/abs/2009.10297>
- [32] Chanchal Kumar Roy and James R Cordy. 2007. A survey on software clone detection research. *Queen's School of computing TR* 541, 115 (2007), 64–68.
- [33] Geoffrey Sawaya, Michael Bentley, Ian Briggs, Ganesh Gopalakrishnan, and Dong H. Ahn. 2017. FLiT: Cross-platform floating-point result-consistency tester and workload. In *2017 IEEE International Symposium on Workload Characterization, IISWC 2017, Seattle, WA, USA, October 1–3, 2017*. IEEE Computer Society, 229–238. doi:10.1109/IISWC.2017.8167780
- [34] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. Adaptive Test Generation Using a Large Language Model. *CoRR* abs/2302.06527 (2023). arXiv:2302.06527 doi:10.48550/ARXIV.2302.06527
- [35] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *Proceedings of the 2012 USENIX Annual Technical Conference, USENIX ATC 2012, Boston, MA, USA, June 13–15, 2012*, Gernot Heiser and Wilson C. Hsieh (Eds.). USENIX Association, 309–318. <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>
- [36] Jackson Vanover, Xuan Deng, and Cindy Rubio-González. 2020. Discovering discrepancies in numerical libraries. In *ISSTA '20: 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, USA, July 18–22, 2020*, Sarfaraz Khurshid and Corina S. Pasareanu (Eds.). ACM, 488–501. doi:10.1145/3395363.3397380
- [37] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14–20, 2024*. ACM, 126:1–126:13. doi:10.1145/3597503.3639121
- [38] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 709–735. doi:10.1145/3689736
- [39] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011, San Jose, CA, USA, June 4–8, 2011*, Mary W. Hall and David A. Padua (Eds.). ACM, 283–294. doi:10.1145/1993498.1993532
- [40] Hengbiao Yu, Xin Yi, Banghu Yin, Fa Li, Zhenbang Chen, and Chun Huang. 2023. Efficient Generation of Floating-Point Inputs for Compiler-Induced Variability. In *IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, Taipa, Macao, March 21–24, 2023*, Tao Zhang, Xin Xia, and Nicole Novielli (Eds.). IEEE, 224–235. doi:10.1109/SANER56733.2023.00030
- [41] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *CoRR* abs/2305.04207 (2023). arXiv:2305.04207 doi:10.48550/ARXIV.2305.04207
- [42] Qirun Zhang, Chengnian Sun, and Zhendong Su. 2017. Skeletal program enumeration for rigorous compiler testing. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18–23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 347–361. doi:10.1145/3062341.3062379